Multiclass Clean-Label Data Poisoning on CIFAR-10

CS 228 — Biometric Security with AI

Garvit Sharma

May 6, 2026

Abstract

We present a clean-label data poisoning attack on a 4-class subset of CIFAR-10, following the *Poison Frogs* methodology [1]. Poison images are crafted to collide in feature space with held-out target images while remaining visually indistinguishable from the base class. Our attack achieves a 10% success rate on a small CNN retrained from scratch, while preserving overall model accuracy (≈71%).

1 Introduction

Clean-label poisoning attacks manipulate training data without changing any labels, making them difficult to detect through standard audits. The goal is to insert crafted "poison" images into the training set so that a model trained on the poisoned data misclassifies specific "target" images at inference time.

We implement the *Poison Frogs* attack [1] on a 4-class subset of CIFAR-10 (Airplane, Automobile, Bird, Cat). The **target class** is Airplane (class 0) and the **base class** is Automobile (class 1). Poison images carry the base class label but are optimised to reside near the target images in the model's feature space.

2 Methodology

2.1 Data Preparation

500 images per class (2 000 total) were sampled from CIFAR-10. Ten target images (Airplane) were held out from training and treated as attack targets. Ten base images (Automobile) served as poison seeds. All poison labels remain 1 (Automobile) throughout — the clean-label property.

2.2 Model Architecture

A lightweight CNN was built with PyTorch:

- **Feature extractor:** 3 convolutional layers (32 / 64 / 128 filters, 3×3 kernels, ReLU + MaxPool)
- **Classifier:** Fully-connected layer (2048→256, ReLU) followed by output layer (256→4)

The 256-dimensional penultimate representation $f(\cdot)$ is used as the feature space for poison optimisation.

2.3 Poison Generation

For each (target t, base b) pair, a poison image x is initialised as b and updated for 100 steps (Adam, lr = 0.05) to minimise:

$$\mathcal{L}(x) = \left\|f(x) - f(t)\right\|_2^2 + \beta\left\|x - b\right\|_F^2, \quad \beta = 0.05 \tag{1}$$

The first term drives x toward t in feature space (*forward step*); the second keeps x visually close to b (*backward step*). Pixel values are clamped to $[0, 1]$ after each update. Fig. 1 shows the evolution of each poison image at iterations 20, 40, 60, 80, and 100.

2.4 Retraining

All 10 poison images (labelled Automobile) are appended to the clean training set. A **fresh model** is trained from scratch for 20 epochs on the poisoned dataset (Adam, lr = 1e-3).

3 Results

3.1 Model Accuracy

Table 1 reports per-class test accuracy before and after poisoning. The poisoned model maintains ≈71% overall accuracy, demonstrating that a defender monitoring validation accuracy cannot detect the attack.

Table 1: Per-class test accuracy: clean vs. poisoned model.

Class	Clean Model	Poisoned Model
Overall	72.83%	70.95%
0 — Airplane	75.10%	70.70%
1 — Automobile	74.50%	96.10%
2 — Bird	75.10%	42.60%
3 — Cat	66.60%	74.40%

3.2 Attack Success Rate

Table 2 shows predictions on the 10 held-out target images before and after poisoning. One target (index 5) was successfully misclassified as Automobile by the poisoned model, yielding a **10% attack success rate**. Notably, Automobile accuracy rose from 74.50% to 96.10%, indicating that the model over-fitted to base-class features introduced by the poisons.

Table 2: Target image predictions before and after poisoning.

Metric	Value
Preds (clean model)	[3, 0, 2, 0, 0, 0, 0, 0, 2, 0]
Preds (poisoned model)	[3, 0, 0, 0, 0, **1**, 0, 0, 2, 0]
Attack Success Rate	10.00%

Fig. 3 visualises the per-class accuracy drop, and Fig. 2 shows the final poison images alongside their base and target counterparts.

4 Discussion

The 10% attack success rate reflects several fundamental challenges of clean-label poisoning in this experimental setting:

1. **Retraining from scratch** forces the network to re-learn all representations, diluting the poisoning signal. Fine-tuning from the clean model retains the original feature structure and is typically more susceptible.
2. **One poison per target.** Prior work [1] achieves significantly higher success rates with 5–50 poison images per target.
3. β **trade-off.** A lower β in Eq. (1) pushes the poison deeper into the target's feature region but reduces visual fidelity; a higher β maintains visual similarity at the cost of feature collision depth.
4. **Model capacity.** Small CNNs form less separable feature spaces than larger networks, making precise feature-space placement harder.

Stealthiness. Despite the modest success rate, the attack remains fully stealthy: all poison images are visually identical to legitimate Automobile photographs, all labels are correct, and overall accuracy drops by only 1.88 percentage points. A human auditor or automated data-quality check would not flag any of the 10 injected images.

5 Visualisations

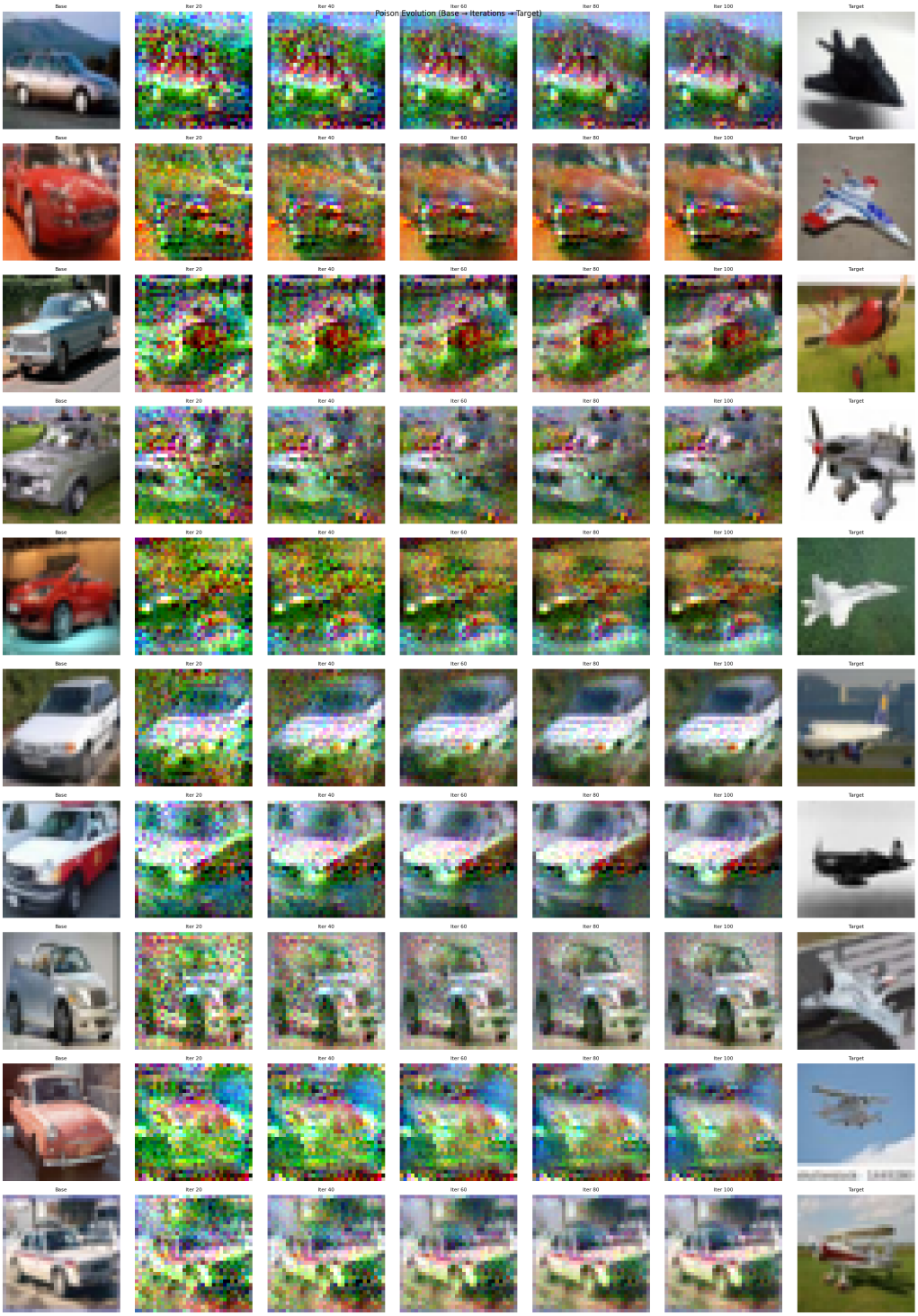

Figure 1: Poison image evolution over 100 optimisation iterations (one row per poison image, 10 rows total). **Leftmost column:** base image (Automobile). **Columns 2–6:** poison at iterations 20, 40, 60, 80, 100. **Rightmost column:** held-out target image (Airplane). The poison remains visually close to the base while its deep features progressively approach those of the target.

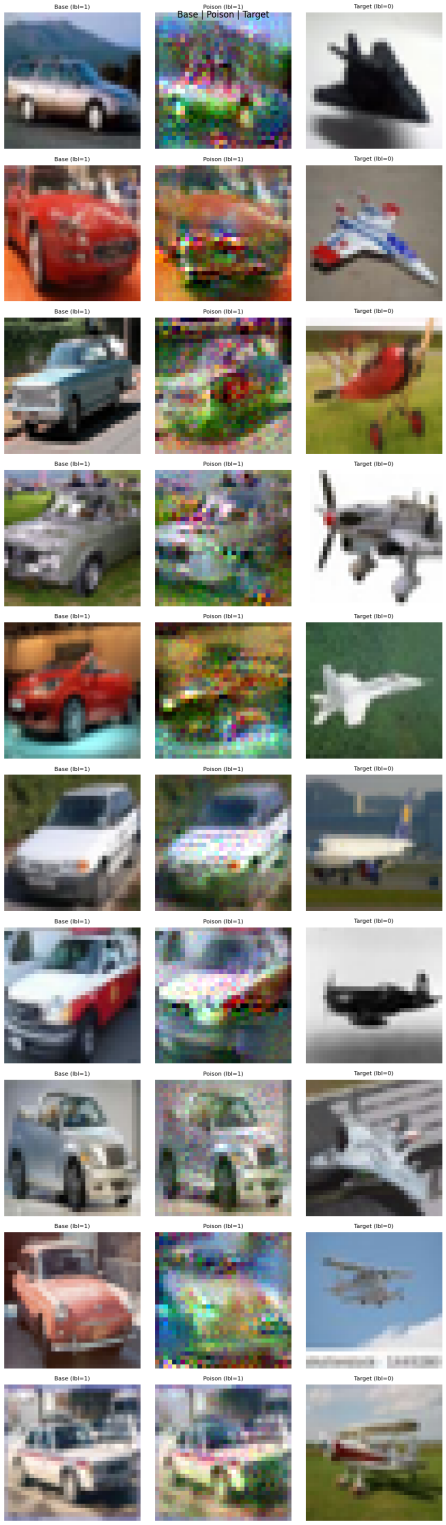

Figure 2: Final comparison: Base | Poison | Target for all 10 pairs. Poison images are visually indistinguishable from legitimate Automobile images yet encode Airplane features in feature space.

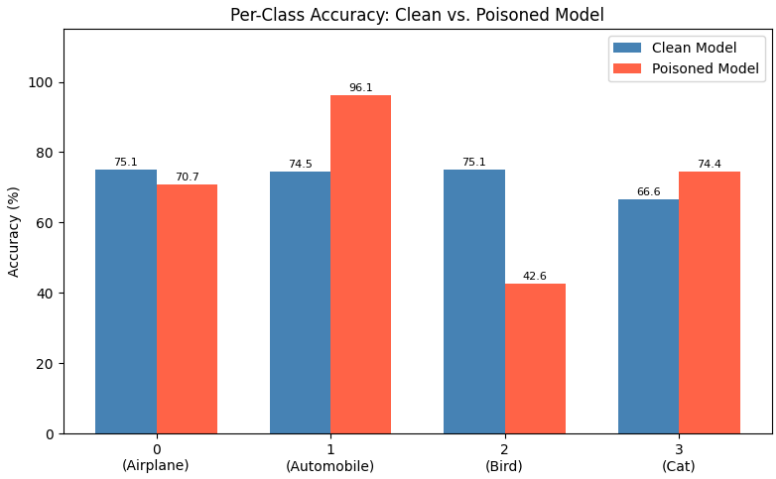

Figure 3: Per-class accuracy of the clean vs. poisoned model on the test set. Automobile accuracy rises sharply (74.50% $\rightarrow$ 96.10%), confirming partial feature collision. Overall accuracy is largely preserved, confirming stealthiness.

References

[1] A. Shafahi, W. R. Huang, M. Najibi, O. Suciu, C. Studer, T. Dumitras, and T. Goldstein, "Poison frogs! Targeted clean-label poisoning attacks on neural networks," in *Proc. NeurIPS*, vol. 31, 2018.